INFOTACT
SOLUTIONS & CO.

Infotact Technical Internship Program: Advanced Python Engineering Project Specifications and Evaluation Criteria

The following document establishes the technical requirements, architectural directives, and sprint-based development roadmaps for three advanced Python Software Engineering projects. Designed for execution within a fast-paced technology environment in Bengaluru, Karnataka, these specifications serve as comprehensive blueprints for backend engineering interns entering the Infotact startup ecosystem.

In 2026, Python engineering extends far beyond basic scripting. Enterprise teams require proficiency in high-concurrency frameworks (FastAPI), asynchronous task queues (Celery/Redis), event-driven architectures (Kafka), and domain-specific data processing. The following projects focus on highly sought-after industry solutions across three critical, untouched niches: Energy & Utilities (Smart Grid Load Balancing), LegalTech (Automated Contract Parsing), and Media & Entertainment (OTT Video Transcoding Pipelines).

This document outlines the requisite tasks, technological paradigms, and evaluation criteria for successful internship completion. It is imperative to note that the assessment of these projects relies heavily on the transparency and consistency of the development lifecycle. **The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.**

Project 1: Energy & Utilities - Smart Grid Load Balancing & Forecasting API

Executive Problem Statement

As the world transitions to renewable energy, the power grid faces massive volatility. Solar and wind energy generation fluctuates by the minute. To prevent blackouts, utility companies must balance the electrical load in real-time by predicting spikes in consumer demand and dynamically routing stored battery power to the grid.

The objective of this project is to build a high-performance Python backend for a Smart Grid Operations Center. The intern will architect an API that ingests continuous telemetry data from simulated smart meters, stores it in a time-series database, and executes background tasks to calculate real-time grid stability metrics.

Business Objectives and Key Performance Indicators

The strategic vision is to ensure 100% grid reliability. Success will be measured by the API's throughput capacity and data ingestion speed. The system must successfully process high-velocity time-series data without dropping payloads, and the background workers must calculate localized load-balancing recommendations within a 5-second SLA (Service Level Agreement).

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Grid Operator	Real-time visibility into neighborhood power consumption.	Monitors a dashboard powered by the API, receiving WebSocket alerts if a specific grid sector exceeds 90% capacity.
Energy Analyst	Access to historical consumption data for forecasting.	Queries the API for aggregated downsampled data (e.g., hourly averages over the last 30 days) to run capacity planning models.

Minimum Viable Product Specifications

The foundational feature is the High-Throughput Ingestion Engine. The intern must utilize FastAPI to build endpoints capable of receiving hundreds of concurrent simulated POST requests containing smart meter readouts (Voltage, Current, Timestamp, Meter ID).
The core deliverable is the Asynchronous Processing Pipeline. Real-time data must be stored in TimescaleDB or InfluxDB. Simultaneously, Celery workers must periodically scan the database to calculate total load per grid sector and flag anomalies.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Web Framework	Python, FastAPI	FastAPI utilizes Starlette and Pydantic to provide asynchronous, non-blocking I/O operations, perfect for high-frequency data ingestion.
Database	TimescaleDB (PostgreSQL)	An extension of PostgreSQL optimized for fast ingest and complex queries on massive time-series datasets.

Task Queue	Celery + Redis	Required for executing background grid calculations without blocking the main API thread.

Four-Week Engineering Roadmap

Week 1: Time-Series Database and FastAPI Setup

The project initiates with configuring the PostgreSQL/TimescaleDB environment using Docker. The intern will design the database schema for the smart meter data. They will then initialize a FastAPI project, implement Pydantic models for data validation, and create the primary ingestion endpoint.

Week 2: Asynchronous Workers and Redis Integration

Week two focuses on decoupling the architecture. The intern will set up Redis and Celery. They will write background tasks that aggregate the raw meter data every 5 minutes to calculate the "Total Megawatt Load" per city zone, saving the aggregated results to a materialized view or separate reporting table.

Week 3: WebSocket Alerts and Complex Queries

The third week transitions to real-time communication. The intern will implement a WebSocket endpoint in FastAPI. If the Celery worker detects that a zone's load exceeds a predefined threshold, it will trigger an event that broadcasts a real-time warning payload over the WebSocket connection to the Grid Operator's dashboard.

Week 4: Dockerization, Optimization, and Documentation

The final sprint is dedicated to production readiness. The intern will write a `docker-compose.yml` file that orchestrates the FastAPI app, Celery worker, Redis, and TimescaleDB into a unified container network. The GitHub repository must be finalized with OpenAPI (Swagger) documentation automatically generated by FastAPI.

Project 2: LegalTech - Automated Contract Parsing & Risk Extraction Engine

Executive Problem Statement

Corporate law firms and enterprise procurement teams manually review thousands of Non-Disclosure Agreements (NDAs) and Master Service Agreements (MSAs) every year. This manual review is slow, expensive, and prone to human error, often resulting in corporations inadvertently agreeing to unfavorable legal clauses (e.g., limitless liability or unfavorable jurisdictions).

The objective of this project is to build an Automated Legal Document Parsing Engine using Python. The intern will architect a Django-based backend that accepts uploaded PDF contracts, extracts the raw text while preserving paragraph structure, and uses Natural Language Processing (NLP) to flag high-risk clauses for legal review.

Business Objectives and Key Performance Indicators

The strategic goal is to reduce contract review time by 70%. Success is measured by the extraction pipeline's accuracy. The system must reliably extract the governing law jurisdiction,

contract duration, and explicitly flag any sentences containing high-risk keywords (e.g., "indemnify", "unlimited liability", "exclusive").

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Paralegal	Rapid sorting of incoming legal documents.	Uploads a batch of 50 PDFs. The system automatically digitizes them and extracts the counterparty names and signing dates.
Senior Counsel	Highlighting hidden risks in 100-page documents.	Reviews the API's output JSON to quickly navigate to the 3 clauses the system flagged as "High Risk."

Minimum Viable Product Specifications

The application foundation is the PDF Ingestion & OCR Pipeline. The intern must utilize libraries like PyMuPDF (Fitz) to extract raw text, ensuring that formatting (headers, bullet points) is logically interpreted.

The core deliverable is the NLP Rule Engine. The intern will use Python's spaCy library to perform Named Entity Recognition (NER) to extract company names and dates, and write complex Regex/NLP rules to categorize the extracted paragraphs into legal clause types (e.g., Confidentiality, Termination).

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend Framework	Python, Django, DRF	Django REST Framework (DRF) provides robust ORM capabilities and a built-in admin panel ideal for managing document metadata.
Document Processing	PyMuPDF (Fitz)	The fastest and most accurate Python library for extracting text, metadata, and images from dense PDFs.

NLP Engine	spaCy (Python)	Industrial-strength NLP in Python, highly optimized for entity extraction and text categorization.

Four-Week Engineering Roadmap

Week 1: Django Setup and Document Management

The intern will initialize a Django project and configure a PostgreSQL database. They will design models for Document, ExtractedClause, and RiskFlag. The primary task is creating a secure API endpoint for uploading PDF files (handling multipart/form-data) and saving them securely to the local file system or a mock S3 bucket.

Week 2: PDF Text Extraction Pipeline

Week two focuses on data extraction. The intern will write utility functions using PyMuPDF to crack open the uploaded PDFs, extract the text page by page, and clean the text (removing whitespace and fixing broken line breaks).

Week 3: NLP Integration and Clause Categorization

The third week introduces intelligence. The intern will integrate spaCy. They will write the logic to parse the cleaned text, isolate the "Governing Law" or "Jurisdiction" clauses, and use NER to identify the contracting parties. They will implement a risk-scoring algorithm that flags specific paragraphs containing dangerous legal jargon.

Week 4: API Serialization and Admin Dashboard

The final week is dedicated to data presentation. The intern will use Django REST Framework serializers to format the extracted document data into a clean, nested JSON response. They will configure the Django Admin panel to allow non-technical users to view the uploaded PDFs and the flagged risks. The repository must contain comprehensive README instructions.

Project 3: Media & Entertainment (OTT) - Video Transcoding & Analytics Pipeline

Executive Problem Statement

When a user uploads a video to a streaming platform (like Netflix or YouTube), the raw file cannot be served directly to viewers. It must be compressed and transcoded into multiple resolutions (1080p, 720p, 480p) to support Adaptive Bitrate Streaming for users with varying internet speeds. Furthermore, platforms need to track real-time analytics on viewer engagement.

This project requires the development of a high-throughput Video Transcoding Pipeline and Streaming Analytics engine. The intern will build a Python backend that orchestrates video processing tasks via FFmpeg, and integrates a message broker to handle incoming streams of simulated viewer metrics.

Business Objectives and Key Performance Indicators

The primary objective is to build a scalable media processing architecture. Success is measured by the system's ability to handle long-running CPU-bound tasks (video compression) without

crashing the main web server. The analytics engine must successfully ingest 1,000+ simulated metric pings per second.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Content Creator	Reliable, fast video uploads with processing status.	Uploads a 100MB MP4 file. The API returns a `job_id`, and the creator polls an endpoint to see the transcoding progress (e.g., "70% Complete").
Platform Admin	Real-time monitoring of video engagement.	Views an analytics dashboard displaying total concurrent viewers and average watch times per video.

Minimum Viable Product Specifications

The foundational feature is the FFmpeg Orchestration Service. The intern must write Python scripts that utilize the `subprocess` module to execute FFmpeg CLI commands, converting raw `.mp4` uploads into optimized, lower-resolution formats.

The core deliverable is the Event-Driven Analytics Ingestion. The intern will deploy Apache Kafka (or Redis Streams) to act as a message broker. A lightweight API will receive JSON payloads representing viewer events (e.g., "Play," "Pause," "Buffer"), which are pushed to Kafka and processed by a Python consumer script in the background.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
API & Routing	Python, Flask	A lightweight microframework perfect for building specific, decoupled microservices (one for upload, one for analytics).

Media Processing	FFmpeg via Python `subprocess`	FFmpeg is the industry standard for media handling; Python acts as the orchestrator to trigger and monitor the transcodes.
Message Broker	Apache Kafka (or Redis Streams)	Essential for handling high-velocity, real-time analytics data streams without dropping events.

Four-Week Engineering Roadmap

Week 1: Flask Upload API and Storage Mocking

The intern will build a Flask application with an endpoint designed to handle large file uploads in chunks to prevent memory overflow. The uploaded video files must be securely saved to a dedicated directory simulating cloud storage. A unique UUID must be assigned to every video asset.

Week 2: Subprocess FFmpeg Transcoding Worker

Week two focuses on CPU-intensive processing. The intern will write a background worker script in Python. When a video is uploaded, the worker uses the `subprocess` module to run FFmpeg commands, generating 720p and 480p versions of the original file, along with generating a thumbnail image from the video's 5-second mark.

Week 3: Kafka Message Broker Integration

The third week targets event streaming. The intern will set up Apache Kafka (via Docker). They will build a new Flask endpoint `/analytics/track` that accepts JSON payloads (e.g., `{"video_id": "123", "event": "buffer", "timestamp": "..."}`) and publishes them to a Kafka topic.

Week 4: Analytics Consumer and Final Polish

The final week closes the loop. The intern will write a standalone Python Kafka Consumer script that subscribes to the analytics topic, aggregates the data (e.g., calculating total views per video), and saves the results to a PostgreSQL database. The GitHub repository must clearly document how to spin up the Kafka/Flask/FFmpeg ecosystem using Docker Compose.

Universal Python Engineering and GitHub Version Control Guidelines

To simulate a rigorous, enterprise-grade development environment indicative of top-tier product companies in Bengaluru, strict adherence to modern software engineering practices is an absolute requirement. The core tenet of this internship program is continuous, transparent, and high-quality contribution.

Mandatory Evaluation Protocol

The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

A project submitted with a compressed commit history—for example, uploading the entire codebase in a massive, monolithic push during the final week—will result in immediate disqualification. Professional Python engineering is a continuous process of iteration, and the version control history must accurately reflect daily script refinement, API testing, and debugging efforts.

Instructions for Tracking Week-Wise Production with Timestamps

To pass the evaluation, interns must utilize GitHub's native tools to explicitly track their weekly production, associating their code commits with timestamped project management artifacts.

1. Setup GitHub Projects (Kanban Board)

At the start of Week 1, the intern must navigate to their repository's **Projects** tab and create a new Kanban board (To Do, In Progress, Done).

- **Mandatory Action:** The intern must break down the 4-week roadmap into individual GitHub Issues (e.g., Issue #1: "Configure TimescaleDB Schema", Issue #2: "Write PyMuPDF Extraction Utility").
- **Timestamping:** GitHub automatically timestamps when an issue is created, moved to "In Progress," and closed. Managers will audit this board to verify a steady, weekly cadence of task completion.

2. Commit Frequency and Semantic Issue Referencing

Commits act as the immutable, timestamped ledger of an engineer's thought process. Interns are expected to commit code frequently (3-5 times per active development day).

- **Mandatory Action:** Every commit message must reference the specific GitHub Issue it solves using semantic messaging.
- **Format Example:** `feat: implemented FFmpeg subprocess for 720p transcoding (fixes #4)`
- By appending `(fixes #4)`, GitHub automatically links the timestamped code commit to the project management board, providing undeniable proof of week-over-week contribution.

3. Python-Specific GitHub Best Practices

Unlike some ecosystems, Python requires strict environment and formatting hygiene within Git:

- **Virtual Environments:** Pushing a `venv/` or `env/` folder to GitHub is an immediate failure. Interns must use a robust `.gitignore` file from Day 1 to exclude cache files (`__pycache__/`, `.pyc`), virtual environments, and `.env` files containing API keys or database passwords.
- **Dependency Tracking:** Every PR must include an updated `requirements.txt` or `pyproject.toml` file to ensure the application environment is reproducible.
- **Branching:** Direct commits to the `main` or `master` branch are forbidden. New endpoints or workers must be developed on feature branches (e.g., `feature/week-3-kafka-integration`) and merged via Pull Requests.

4. Automated CI/CD Code Quality Timestamping

To prove the codebase was continuously maintained, interns must implement a basic GitHub Actions workflow (`.github/workflows/python-app.yml`).

- **Mandatory Action:** The CI pipeline must run a linter (like `flake8` or `black`) and execute any written `pytest` unit tests on every push.

- **Evaluation Check:** The GitHub Actions tab provides an immutable execution log with exact timestamps. Managers will verify that the CI pipeline was triggered consistently, proving that the intern was writing clean, PEP-8 compliant Python code throughout the 4 weeks, not just at the final deadline.

By strictly enforcing these GitHub tracking specifications, the program ensures that deliverables transcend mere functional prototypes, transforming into robust, highly maintainable Python applications built with elite, verifiable engineering standards.